



FACULTAD DE INGENIERÍA DE SISTEMAS

DOCENTE:
LUIS MURILLO

INTEGRANTES:

FRANKLIN MIRANDA
SEBASTIAN BARRAGAN
LUISA CHACON
JOSE RAMOS
JUAN RODRIGUEZ

UNIVERSIDAD DEL SINU
SECCIONAL CARTAGENA

10/08/2026

SIMULACIÓN SEMÁFORO CON ARDUINO UNO

Control secuencial de un componente semáforo 3 leds en 1, desarrollado en el IDE arduino, simulado en tinkercad para luego realizar dicho laboratorio funcional con los elementos.

INTRODUCCIÓN

Este laboratorio consiste en utilizar la lógica de un semáforo real donde utilizamos **arduino uno** así como en la práctica del laboratorio anterior y una protoboard. Objetivo de ejecutar la secuencia de un semáforo respetando los colores de seguridad rojo para detenerse, amarillo precaucion y verde continuar.

Se realizó una simulación digital y luego pasamos a la parte física de los componentes pero en este caso reemplazamos los leds por un módulo completo parecido a un semáforo respetando la estética y los protocolos de los colores.

MATERIALES

- 1 x Placa Arduino UNO
- 1 x Protoboard (breadboard)
- Módulo tres pines con LEDS incluido o módulo semáforo
- Cables jumper macho-macho
- 1 x Cable USB (alimentación y carga del programa) • PC con Arduino IDE instalado

DIAGRAMA Y SIMULACIÓN EN TINKERCAD

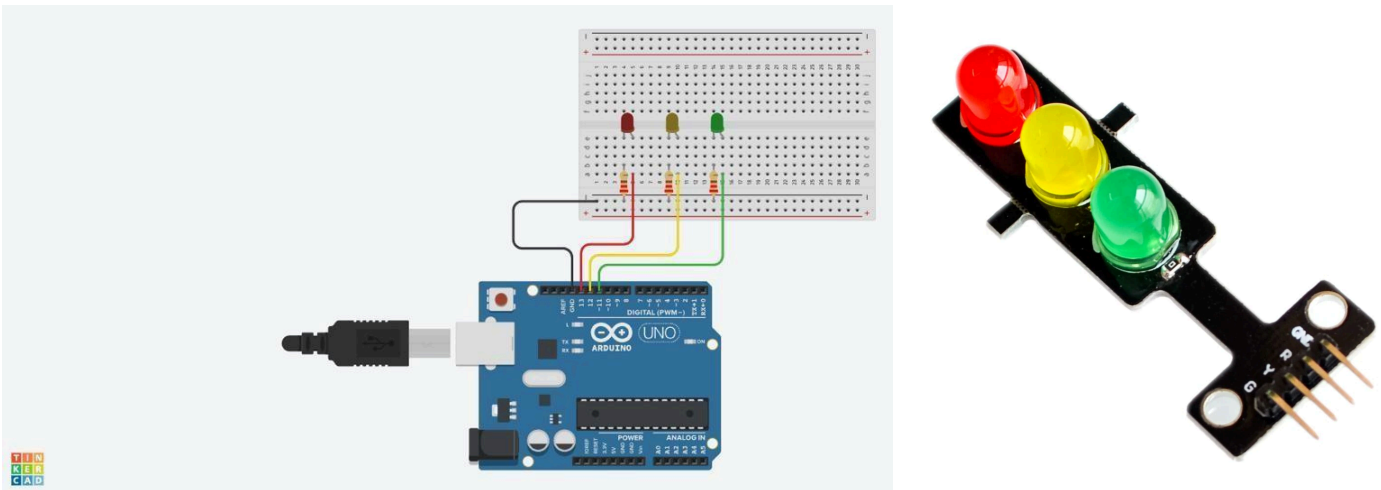


Figura 1. Simulación del circuito en Tinkercad: LED rojo, amarillo y verde conectados a los pines digitales 13, 12 y 11 del Arduino UNO

Antes de ensamblar el circuito físico, la configuración fue diseñada y probada en tinkercad. En el diagrama vemos los leds montados en la placa pero estos serán reemplazados por el módulo del semáforo que trae su propio GND solo es conectar la salida de los jumpers a la protoboard en la línea de conexión de los pines del módulo del semáforo

CONEXIÓN DEL CIRCUITO

Componente	Conexión en el Arduino
LED Rojo (ánodo, vía resistencia)	Pin digital 13
LED Amarillo (ánodo, vía resistencia)	Pin digital 12
LED Verde (ánodo, vía resistencia)	Pin digital 11
Cátodos de los 3 LEDs (riel negativo protoboard)	Pin GND

CÓDIGO FUENTE (ARDUINO IDE)

El programa establece tres variables para los pines de los LEDs, los designa como salida (OUTPUT) durante la función setup(), y en la función loop() activa cada LED de forma individual por un periodo específico utilizando delay, repitiendo este proceso indefinidamente.

```
int ledRojo = 13;
int ledAmarillo = 12;
int ledVerde = 11;

void setup() {
  pinMode(ledRojo, OUTPUT);
  pinMode(ledAmarillo, OUTPUT);
  pinMode(ledVerde, OUTPUT);
}

void loop() {
  //LED ROJO
  digitalWrite(ledRojo, HIGH);
  digitalWrite(ledAmarillo, LOW);
  digitalWrite(ledVerde, LOW);
  delay(60000);

  //LED AMARILLO
  digitalWrite(ledRojo, LOW);
  digitalWrite(ledAmarillo, HIGH);
  digitalWrite(ledVerde, LOW);
  delay(5000);

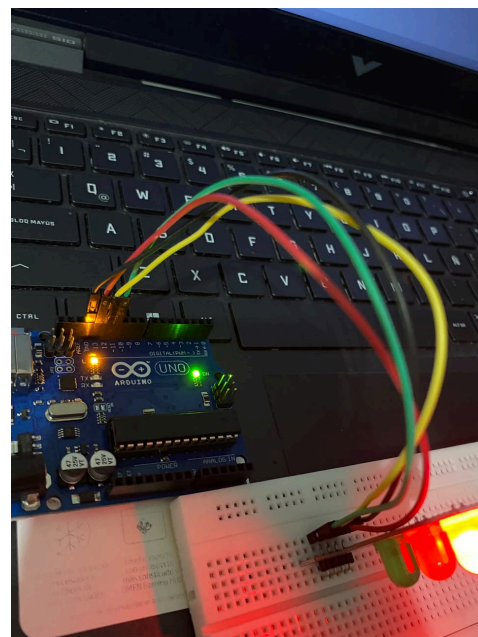
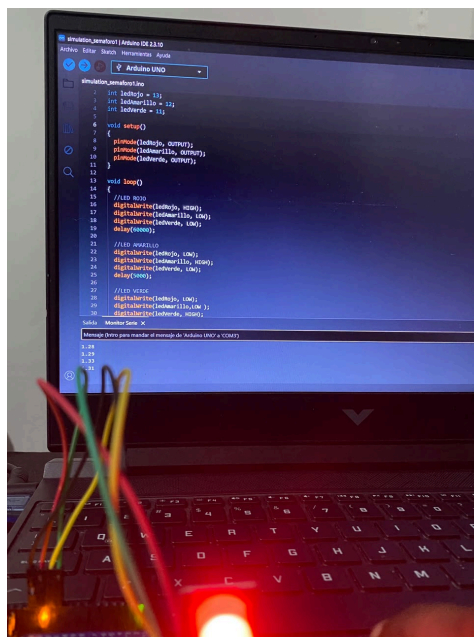
  //LED VERDE
```

```
digitalWrite(ledRojo, LOW);
digitalWrite(ledAmarillo, LOW);
digitalWrite(ledVerde, HIGH);
delay(30000);
}
```

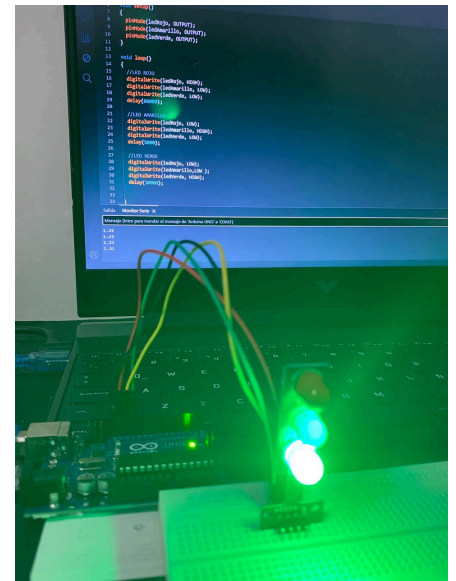
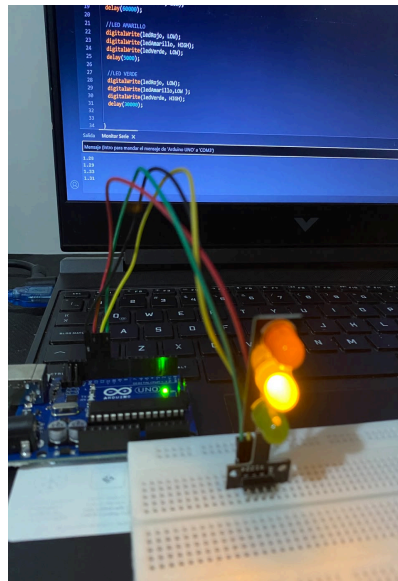
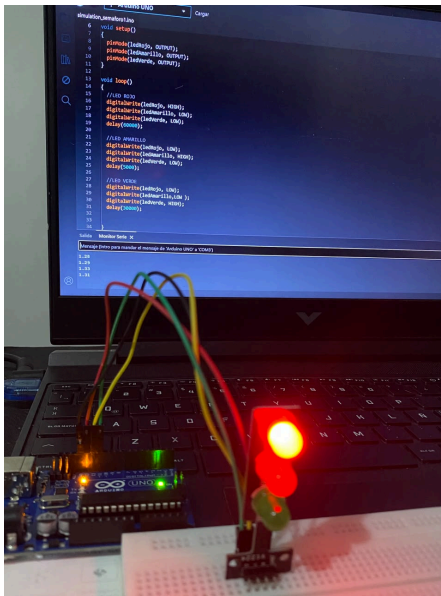
LÓGICA DE FUNCIONAMIENTO (TEMPORIZACIÓN)

Etapas	LED activo	Duración	Significado
1	Rojo	60 s (60000 ms)	Alto / detenerse
2	Amarillo	5 s (5000 ms)	Precaución
3	Verde	30 s (30000 ms)	Siga / vía libre

Una vez que se validó la simulación, se construyó el circuito físico en una protoboard, replicando fielmente las mismas conexiones (pines 13, 12 y 11), y se transfirió el mismo código al Arduino UNO real a través del cable USB. A continuación, se presentan imágenes que ilustran las tres etapas del ciclo en el montaje real:



Fase 1 montaje de los componentes y carga del código arduino



Fase 2 visualización de la secuencia del semáforo mostrando los protocolos de precaución

RESULTADOS Y CONCLUSIÓN

El circuito funcionó correctamente en la simulación y también en la aplicación real utilizando el módulo de semáforo conectado a Arduino con cables de puente. El tiempo y el orden de las luces roja, amarilla y verde estaban en línea con la lógica programada de un semáforo real y tanto los módulos como el código eran correctos.

Posibles mejoras para una próxima versión:

- Reemplazar el retraso con temporización usando `millis()` para que Arduino pueda realizar otras tareas mientras el semáforo está funcionando
- Agregar un módulo peatonal o semáforo que esté sincronizado con el ciclo del semáforo de vehículos. Usar `const int` para definir los pines de conexión ya que permanecen constantes mientras se ejecuta.
- Si deseas habilitar que cada luz se configure en tiempo mediante un potenciómetro o variables en el código, es decir, a través del código.